

NGUYEN HOAI NAM

FRESHER UNITY GAME DEVELOPER

Ho Chi Minh City, Vietnam • (+84) 0384122084 • codenamtap@gmail.com • nhoinam-gamedev.vercel.app • github.com/hoainamcode22 • nguyenhoainam.itch.io

PROFESSIONAL SUMMARY

Proactive **Fresher Unity Game Developer** and final-year Information Technology student at UTH with **1+ year of hands-on solo indie game engineering experience** in C# and Unity. Shipped a full farming-simulation title to WebGL and Android, then profiled and rebuilt its hot paths to reach **0 B/frame GC allocations, zero fatal crashes on iOS Safari**, and a steady **60 FPS** on mid-range mobile hardware. Comfortable owning a feature end to end: gameplay architecture, custom internal editor tooling, client-side profiling, and multiplatform build configuration.

CORE TECHNICAL SKILLS

Engine & Languages:	Unity Engine (2D/3D), C# (.NET), C++ (algorithms & low-latency search), clean OOP, event-driven architecture, component design
Gameplay Systems:	ScriptableObject data pipelines, Finite State Machines, recipe/crafting engines, inventory & economy, kinematic 2D physics, raycasting
Internal Tooling:	Custom Unity Editor scripting (CustomEditor, EditorWindow) for automated 100+ tile coordinate alignment and obstacle generation
Profiling & Optimization:	Unity Profiler , custom in-game F4 diagnostic overlay, Object Pooling , zero-alloc physics (NonAlloc buffers), draw-call batching, Sprite Atlas, canvas isolation
WebGL & Memory:	iOS WebKit OOM crash resolution, linear Wasm heap growth (16 MB steps), DPR clamping, ASTC 6x6 texture compression, 1080p resolution clamping
Build & Infrastructure:	WebGL (Brotli/Gzip), Android Gradle APK, DOTween, TextMeshPro, Input System, binary serialization, Git / GitHub
Supporting Stack:	Basic 3D modeling (Blender), web fundamentals (HTML/CSS/JS), relational DB (PostgreSQL) English: TOEIC 600

KEY COMPETITIVE ACHIEVEMENT

Nexon Youth Programming Challenge (NYPC 2026) – Master Track Top 86 / 1,570 Competitors (Top 5%) Jun 2026 – Aug 2026

- Team: **3 Chàng Linh Ngự Lâm** (3 members) • Role: **Team Leader & Core Algorithm Architect** • Strategy AI ("NEXT NATION") • **C++ • 7 NYPC Competency Badges**
- Tactical search engine (personal core contribution):** Architected the C++ dynamic heuristic evaluation for a 1v1 turn-based strategy game, balancing gold logistics, building expansion and troop combat across a 200-turn horizon under a strict **100 ms per-turn SLA** with **0 timeout penalties**.
 - Team leadership & architecture scaffolding:** Led a 3-engineer team, designed the modular framework (standard I/O command pipeline, turn-validation engine, state serialization), and split ownership between data infrastructure and combat search trees.
 - Competitive consistency:** Reached **Top 1 and Top 10** in preliminary sparring rounds and held **Top 10 to Top 30** through live qualifiers before finishing **Top 86 / 1,570**.

SELECTED TECHNICAL & OPTIMIZATION IMPACT

- Crash elimination:** Removed a **~95% white-screen crash rate** on **iOS Safari WebGL** caused by the WebKit per-tab memory ceiling and IndexedDB iframe quota exceptions, reaching **100% stable boot** with no WebGL context loss.
- CPU main-thread frame time:** Cut peak interaction frame cost from **~150 ms to 10–15 ms**; ScriptRunBehaviourUpdate fell from **~91 ms to 6–10 ms** and total PlayerLoop from **~104 ms to 11–14 ms**.
- Zero-allocation gameplay:** Converted every hot-path physics query to NonAlloc buffers, taking steady-state GC from **50–200 KB/s to 0 B** and removing recurring collection stutter.
- GPU VRAM:** Reduced VRAM by **~75% (520–680 MB to 130–165 MB)** using hardware ASTC 6x6 compression, Sprite Atlas ganging, and device-pixel-ratio clamping from 3.0x to 1.5x on Retina displays.
- Memory & Wasm heap:** Lowered peak WebAssembly heap by **~65% (750 MB–1.1 GB to 180–320 MB)** via linear 16 MB growth steps, a 512 MB safety cap, and disabled texture Read/Write.
- Boot time & payload:** Accelerated first-scene boot **3.5x (22 s to under 5 s)** and cut network payload **~72% (150 MB to 38–44 MB)** with Brotli streaming compression.
- Frame rate:** Raised mid-range mobile performance from **24–42 FPS to a solid 58–60 FPS**, with measurably cooler thermals and **50–70% lower GPU fillrate**.

GAMEPLAY ENGINEERING & INDIE PROJECTS (100% SOLO DEVELOPMENT)

Cooking-Game-2D (FarmHn) — Solo Indie Game Developer (1 year · end-to-end ownership) Aug 2023 – Aug 2024

Unity, C#, DOTween, ScriptableObject, grid system, object pooling, WebGL & Android APK (198 MB) • github.com/hoainamcode22/Cooking-Game-2D • nguyenhoainam.itch.io/farmhn

- Core systems & culinary economy:** Designed and built the full agricultural simulation and recipe-crafting engine end to end — ScriptableObject-driven recipe trees with ingredient validation and tier unlocks, crop growth cycles with drag-and-drop planting, warehouse storage, livestock FSMs, food-processing factories, and multi-tier train/ship delivery fulfilment.
- Internal Unity Editor tooling:** Wrote custom CustomEditor and EditorWindow tools that automate coordinate alignment and obstacle generation across 100+ tiles, cutting level-layout turnaround by **~40%**.
- iOS WebKit crash investigation:** Traced a hard failure where the build consistently died at the 90% load mark on iPhone and iPad. Root-caused it to three compounding limits — the Safari per-tab memory ceiling reached by geometric heap growth, a Retina 3x framebuffer inflating VRAM past the mobile GPU budget, and a QuotaExceededError from writing a 150 MB bundle into a third-party iframe capped at 50 MB of IndexedDB. Fixed all three and took iOS from **unplayable to 100% stable**.
- Adaptive device tiering:** Built WebGLTierScaler.cs to classify constrained iOS WebKit, standard Android, and desktop at runtime, then tune device pixel ratio, soft particles, async upload buffer size (8/16/32 MB) and particle raycast budget per tier.
- Texture & fillrate profiling:** Re-encoded 1,000+ uncompressed RGBA32 source textures to hardware ASTC 6x6 (~16 MB to ~2.7 MB per 2048×2048 sheet), right-sized max texture dimensions per asset class, disabled Read/Write and 2D mipmaps, and clamped render height to 1080p — saving roughly **80% VRAM** and **50–70% GPU fillrate**.
- Zero-alloc physics & pooling across 23 systems:** Replaced LinecastAll and OverlapCircleAll with preallocated NonAlloc buffers in the sickle, planting and plot-input controllers; pooled harvest text, slash VFX and train smoke puffs instead of instantiating; and prewarmed composite pools to remove a **50–80 ms hitch** on every harvest swing.
- Update-loop and canvas hygiene:** Eliminated a recurring FindObjectsByType sweep over 6,000+ scene objects that ran every 0.35–2 s, added Renderer.isVisible culling with 30 Hz phase-staggered ticks for ambient foliage, gated 26 dormant background subsystems behind a feature flag, and isolated coin/gem fly effects onto their own overrideSorting sub-canvas so they no longer dirty a 950-element HUD canvas.
- Data persistence:** Serializing dynamic state across 100+ tiles caused 2.8 s load spikes; refactored to a compact binary-struct format with ID mapping, cutting save size **65%** and load time to **under 0.6 s**.
- Diagnostics:** Shipped an in-game realtime profiler (F4 overlay) exposing FPS, frame time, managed heap and per-frame allocation, so regressions surface during play instead of in review.

SkyBound — Solo Gameplay Developer & Physics Prototyper (rapid prototype · 1 week)

Sep 2024

Unity 2D, kinematic physics, virtual touch controls, WebGL & Android (41 MB) • github.com/hoainamcode22/Game_Dao_Vang • nguyenhoainam.itch.io/sky

- Kinematic controller & springboard trajectory:** Wrote a custom kinematic character controller with hand-tuned gravity and velocity curves and downward raycast ground detection, then layered trajectory force overrides on interactive spring launchers using parabolic apex calculations, so launch height stays predictable regardless of entry speed.
- Input latency & physics tunneling:** Replaced per-frame touch polling with continuous 2D collision sweeps in FixedUpdate and cached touch pointer IDs, removing tunneling through thin platforms at high fall speed and cutting touch response from **~80 ms to under 16 ms (single frame)**.
- Modular prefab architecture:** Built reusable prefab chunks for floating islands, hazards and collectibles so new levels compose from tested pieces, shipping a lightweight **41 MB** build at a steady **60 FPS** across WebGL and Android.
- Scope discipline:** Took the prototype from empty project to publicly playable build in **one week**, deliberately limiting scope to validate the physics feel before investing in content.

EDUCATION

University of Transport Ho Chi Minh City (UTH)

Oct 2022 – Expected 2026

Bachelor of Science in Information Technology • Final-year student